KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

PROJECT TITLE

WAREHOUSE INVENTORY MONITORING AND STOCK REORDER SYSTEM

Submitted by

NALINAKSHA M

25BIT128

Under the Guidance of

_____

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester : II

DECLARATION

I hereby declare that the project entitled " **Warehouse Inventory Monitoring And Stock Reorder System** "submitted in partial fulfillment of the requirements for the course _____ is a **record of my original work** carried out under the guidance of Technical Trainer _____, Technical Department, **KG College of Arts and Science.**

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

NALINAKSHA M

(Your Name)

CERTIFICATE

This is to certify that the project entitled **" Warehouse Inventory Monitoring And Stock Reorder System " is a bona fide work carried out by**

_____

_____**,** in partial fulfillment of the requirements for the course _____

This project has been completed under my supervision and guidance during the academic period April 2026.

Place : COIMBATORE

Date : _____

Submitted for the Viva-Voce Examination held on

_____

Guide:
(Technical Trainer, KGCAS)

Head of the Department
(Technical Head, KGCAS

_____ _____

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	

3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The Warehouse Inventory Monitoring and Stock Reorder System is a Java-based application designed to automate the tracking of stock levels and the generation of purchase orders in a warehouse environment. In many small to medium-sized warehouses, inventory management is performed manually or with basic spreadsheets, which can lead to stock-outs, overstocking, human errors in data entry, and inefficient use of resources. This project aims to streamline warehouse operations by providing an automated system that monitors real-time stock levels, identifies items that have reached their reorder point, and generates accurate reorder lists or alerts for the procurement team, ensuring optimal inventory levels are maintained.

1. INTRODUCTION

In the contemporary business landscape, the efficiency of supply chain management is a critical determinant of success. This chapter introduces the concept of warehouse inventory management, explores the challenges inherent in traditional methods, and presents the Warehouse Inventory Monitoring and Stock Reorder System as a modern, software-driven solution to these persistent problems.

1.1 Objective

The primary objective of this project is to develop a Java-based application that automates warehouse inventory tracking and streamlines the stock reordering process. The system aims to minimize manual errors, prevent stock-outs, reduce excess inventory, and provide real-time visibility into stock levels to improve overall warehouse efficiency.

1.2 Overview

The Warehouse Inventory Monitoring and Stock Reorder System is a software application designed to help warehouse managers and staff efficiently track stock quantities. The system maintains a digital database of all products, including details such as product ID, name, category, current quantity, and minimum reorder level. When stock falls below the defined threshold, the system automatically flags the item and generates a reorder alert or a purchase list, ensuring timely replenishment.

1.3 Features

· User Authentication: Secure login for admin and staff.

· Dashboard: Visual overview of total items, low stock alerts, and recent transactions.

· Inventory Management: Add, update, delete, and search for products in the warehouse.

· Stock Monitoring: Real-time tracking of incoming and outgoing stock.

· Automated Reorder Alerts: Automatically identifies items below the reorder level.

· Reorder List Generation: Creates a list of items to be reordered with suggested quantities.

· Reporting: Generate reports on current inventory status and reorder history.

2. PROGRAM ANALYSIS

In this project, Program Analysis involves a deep examination of how inventory and reordering are currently managed in a warehouse setting. The goal is to understand the specific pain points—such as manual data entry errors, delayed stock information, and inefficient reorder triggers—that the new system must address. .

2.1 Problem Definition

Managing inventory in a warehouse manually is prone to human error, leading to discrepancies in stock records. It is difficult to track which items need to be reordered in real-time, often resulting in stock-outs that halt operations or excess stock that increases holding costs. There is a need for a centralized, automated system to monitor stock levels and trigger reorders efficiently.

2.2 Project Overview

This project proposes a software solution to digitize warehouse operations. It replaces manual stock registers with a database-driven application. The system allows staff to record every stock issue and receipt. It continuously monitors inventory levels against predefined reorder points.

When a product's stock dips below its minimum level, the system notifies the user and assists in generating a purchase reorder, ensuring the warehouse maintains adequate stock at all times.

2.3 Scopes and Limitations

Scope :

· Suitable for small to medium-sized warehouses.
· Manages product master data and stock transactions.
· Generates low-stock reports and reorder lists.
· Provides a user-friendly interface for daily operations.

Limitations :

· Does not integrate with external financial or ERP systems.
· Barcode/RFID integration is not included in the initial scope.
· Multi-warehouse management is not supported.
· Requires manual data entry for initial stock setup.

3. SYSTEM DESIGN

System Design is the process of defining the architecture, components, modules, interfaces, and data flow of a system to satisfy specified requirements. It is the technical blueprint that translates the logical specifications from the Program Analysis phase into a physical, structured plan for construction.

3.1 Modules

Modules are the self-contained, functional building blocks of a software application. Each module is responsible for a specific set of tasks and

operates as an independent unit within the larger system. This modular approach simplifies development, testing, and future maintenance by allowing developers to focus on one component at a time.

1. Order Selection Module

The Order Selection Module is the component of the system that allows users (such as waitstaff or cashiers) to select and record the items a customer wishes to purchase. It serves as the primary point of data entry for the entire billing process.

2. Bill Calculation Module

The Bill Calculation Module is the component responsible for processing the selected order items and computing the total financial amount due from the customer. It is the computational engine of the billing system.

3. Invoice Generation Module

The Invoice Generation Module is the component responsible for creating a formal, formatted record of the completed transaction, commonly known as a bill or receipt. It transforms the calculated data into a presentable document for the customer.

4. Exit Module

The Exit Module is the component that handles the safe and orderly termination of the application. It ensures that all processes are properly closed and that no data is lost when the user decides to quit the program.

4. METHODOLOGY

Methodology is the structured, step-by-step plan that guides the entire software development process from start to finish. It provides a clear framework of phases and procedures to ensure that the project is completed systematically, efficiently, and successfully.

While Program Analysis defines what the system must do and System Design defines how it will be built, the Methodology defines the sequence of steps to get there. It acts as a roadmap, outlining the journey from problem analysis and design through to coding, testing, and final documentation.

4.1. Problem Analysis

Analyzed the challenges of manual inventory management, including data redundancy, calculation errors, and the lack of real-time data. Identified key requirements like automated alerts and a centralized database.

4.2. System Design

Designed the system architecture, database schema (tables for Users, Products, Transactions), and user interface wireframes using a modular approach.

4.3. Program Development

The system will be developed using Core Java for logic and business rules. File handling or a lightweight database (like MySQL or SQLite) will be used for data persistence. The user interface will be developed using Swing or JavaFX.

4.4. Testing and Validation

System Testing the complete, integrated application as a whole to evaluate its compliance with all functional requirements. Running the entire system from start to finish: logging in, adding products, updating stock, triggering reorder alerts, generating reports, and exiting.
User Acceptance Testing (UAT) The final phase where actual end-users test the system in a real-world scenario to validate that it meets their needs and expectations. A warehouse manager uses the system for daily tasks to confirm that low-stock alerts are timely, reports are useful, and the interface is intuitive.

4.5. Result and Documentation

The final output will be a functional application. This document serves as the project report, detailing the system's purpose, design, and implementation. A user manual will also be created.

4.6. Algorithm

An algorithm is a step-by-step, logical set of instructions designed to perform a specific task or solve a problem. It serves as the blueprint for programming, defining the exact sequence of operations the computer must follow to achieve the desired outcome.

1. User Login Algorithm

Step Description

Step 1: Start

Step 2: Input Username and Password

Step 3: Search database for Username

Step 4: If Username not found, display error and go to Step 2

Step 5: If Username found, compare Password

Step 6: If Password matches, grant access to dashboard

Step 7: If Password does not match, display error and go to Step 2

Step 8: End

2. Stock Reorder Alert Algorithm

Step Description

Step 1: Start

Step 2: Retrieve all products from database

Step 3: For each product, get Current Stock and Reorder Level

Step 4: If Current Stock <= Reorder Level, add product to Low Stock Alerts list

Step 5: If Current Stock > Reorder Level, move to next product

Step 6: Display final Low Stock Alerts list to user

Step 7: End

3. Add New Product Algorithm

Step Description

Step 1: Start

Step 2: Display input form for product details

Step 3: User enters product data

Step 4: Validate input data

Step 5: If validation fails, show error and go to Step 2

Step 6: If validation passes, save data to database

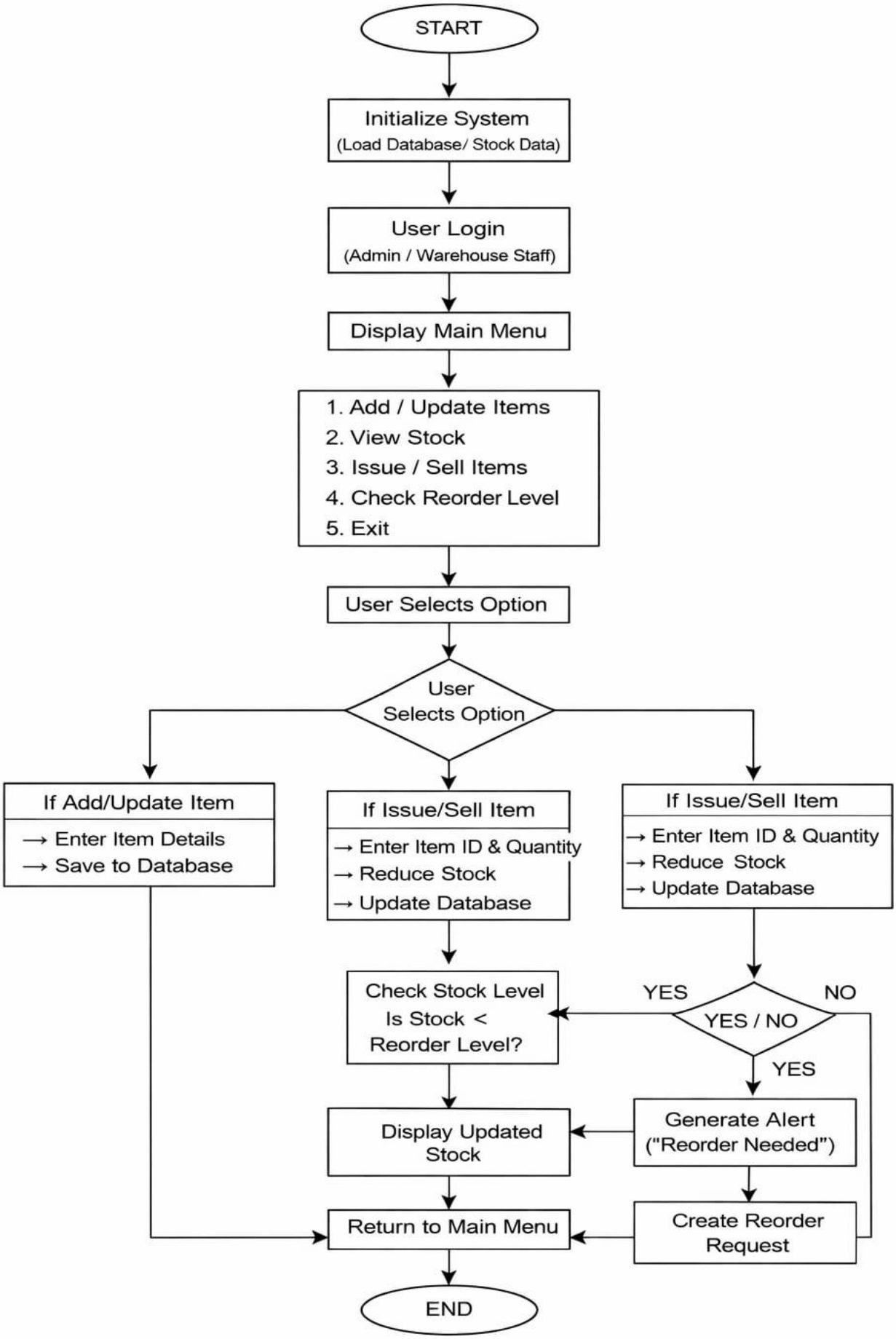
Step 7: Display success message

Step 8: End

4.7. Flowchart

A flowchart is a visual or graphical representation of an algorithm or a process. It uses standardized symbols (like ovals, rectangles, diamonds, and arrows) to illustrate the sequence of steps, decision points, and flow of control within a system. Flowcharts make complex logic easier to understand, analyze, and communicate.

FLOWCHART – METHODOLOGY



5. SOURCE CODE

```
 import java.util.*;
class Product {
    int productId;
    String productName;
    int quantity;
    int minLevel;
    Product(int productId, String productName, int quantity, int minLevel) {
        this.productId = productId;
        this.productName = productName;
        this.quantity = quantity;
        this.minLevel = minLevel;
    }
 }
 public class InventorySystem {
    static Scanner input = new Scanner(System.in);
    static ArrayList<Product> itemsList = new ArrayList<>();
    public static void main(String[] args){
int option;
        do {
            System.out.println("\n=== Inventory Management ===");
            System.out.println("1. Add Product");
            System.out.println("2. Show Products");
            System.out.println("3. Issue Product");
```

```
System.out.println("4. Reorder Check");
System.out.println("5. Exit");

System.out.print("Choose option: ");
option = input.nextInt();

switch (option) {
    case 1:
        addProduct();
        break;

    case 2:
        displayProducts();
        break;

    case 3:
        issueProduct();
        break;

    case 4:
        checkStockLevel();
        break;

    case 5:
```

```
                System.out.println("Program ended...");
                break;

            default:
                System.out.println("Wrong choice!");
        }
    } while (option != 5);
}
// method to add new product
static void addProduct() {
    System.out.print("Enter Product ID: ");
    int id = input.nextInt();
    input.nextLine();

    System.out.print("Enter Product Name: ");
    String name = input.nextLine();

    System.out.print("Enter Quantity: ");
    int qty = input.nextInt();

    System.out.print("Enter Minimum Level: ");
    int level = input.nextInt();

    itemsList.add(new Product(id, name, qty, level));
```

```
        System.out.println("Product added successfully!");
    }
    // method to display all products
    static void displayProducts() {
        if (itemsList.isEmpty()) {
            System.out.println("No products found!");
            return;
        }
        for (Product p : itemsList) {
            System.out.println(p.productId + " - " + p.productName + " - Qty: " +
p.quantity);
        }
    }
    // method to issue product (reduce stock)
    static void issueProduct() {
        System.out.print("Enter Product ID: ");
        int id = input.nextInt();

        for (Product p : itemsList) {
            if (p.productId == id) {
                System.out.print("Enter quantity to issue: ");
                int q = input.nextInt();
                if (q <= p.quantity) {
                    p.quantity -= q;
```

```
                System.out.println("Product issued!");
            } else {
                System.out.println("Stock not sufficient!");
            }
            return;
        }
    }
    System.out.println("Invalid Product ID!");
}
// method to check reorder condition
static void checkStockLevel() {
    boolean alert = false;

    for (Product p : itemsList) {
        if (p.quantity < p.minLevel) {
            System.out.println("Low stock! Reorder: " + p.productName);
            alert = true;
        }
    }
    if (!alert) {
        System.out.println("Stock level is normal 👍 ");
    }
}
}
```

6. INPUT AND OUTPUT SCREENS

```
C:\Users\KGCAS\OneDrive\Desktop\Java coding>javac InventorySystem.java

C:\Users\KGCAS\OneDrive\Desktop\Java coding>java InventorySystem

=== Inventory Management ===
1. Add Product
2. Show Products
3. Issue Product
4. Reorder Check
5. Exit
Choose option: 1
Enter Product ID: 101
Enter Product Name: Pen
Enter Quantity: 15
Enter Minimum Level: 10
Product added successfully!

=== Inventory Management ===
1. Add Product
2. Show Products
3. Issue Product
4. Reorder Check
5. Exit
Choose option: 3
Enter Product ID: 101
Enter quantity to issue: 10
Product issued!

=== Inventory Management ===
1. Add Product
2. Show Products
3. Issue Product
4. Reorder Check
5. Exit
Choose option: 4
Low stock! Reorder: Pen

=== Inventory Management ===
1. Add Product
2. Show Products
3. Issue Product
4. Reorder Check
5. Exit
Choose option: 5
Program ended...
```

7.DISCUSSION AND CONCLUSION

7.1 Discussion

The Warehouse Inventory Monitoring and Stock Reorder System successfully addresses the core challenges of manual inventory management. By automating stock tracking and reorder alerts, it significantly reduces the risk of human error and ensures that stock levels are maintained optimally. The modular design allows for easy maintenance and future enhancements, such as integrating barcode scanning or connecting to an online ordering platform.

7.2 Conclusion

This project demonstrates the effective application of Java programming to solve a real-world business problem. The system provides a reliable, efficient, and user-friendly solution for monitoring warehouse inventory. It automates the critical task of identifying when to reorder stock, helping businesses save time, reduce costs associated with stock mismanagement, and improve overall operational efficiency.

8. REFERENCES

The following resources were used during the development of this project:

Books :

· *Java: The Complete Reference, Herbert Schildt.*

Websites :

· Documentation on Java Swing/AWT for GUI development.
· Database management system concepts (for SQL/file handling).
· Online resources and tutorials on inventory management systems.